Software Engineering & Project Management (SEPM)

Energy & Utility Management System (EUMS)

Billing, metering and customer service for electricity, water and gas

Database Design

Group number	Group 3
Course and section	Software Engineering & Project Management (SEPM), CSEF
Date	October 2026

#	Name	Registration number
1	Sumit P Surendran	20243590
2	Sreethan Reddy	20243550
3	Sushrut Lad	20243551
4	Viswajith	20243559

1. Database Design

1.1 ER diagram and data model

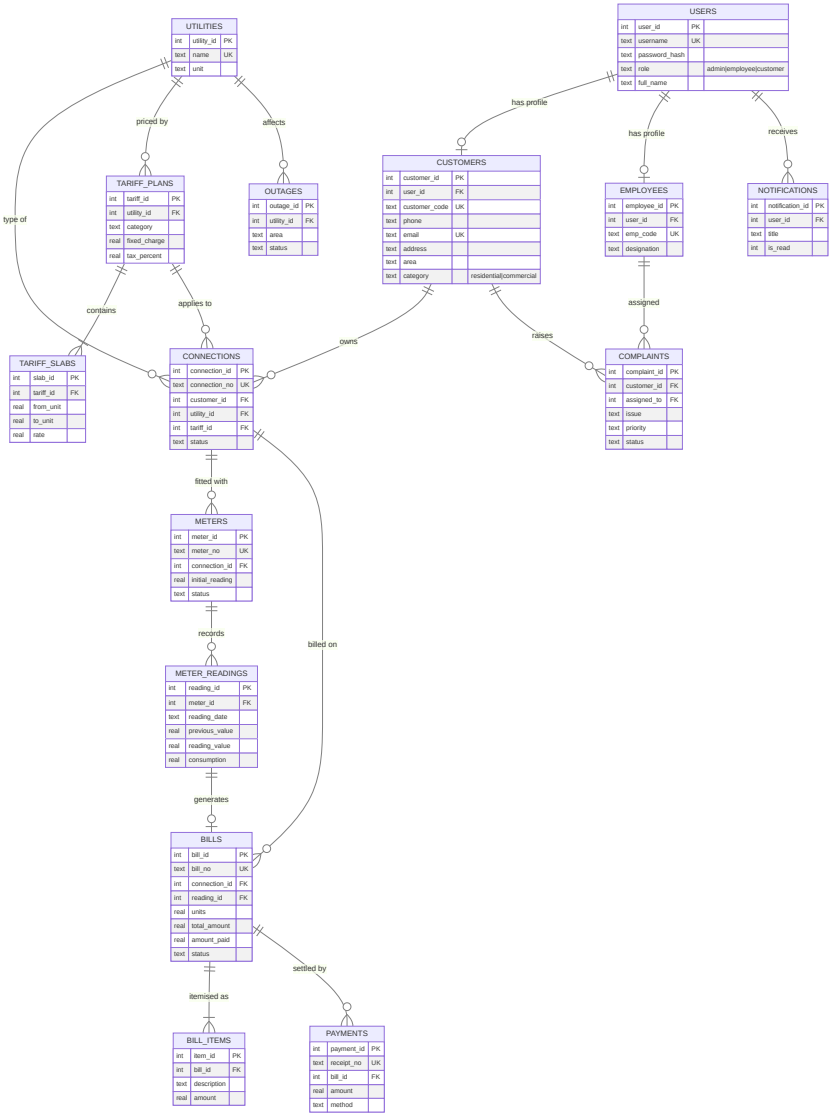


Figure 1: Entity–relationship diagram (crow's-foot notation)

Table	Purpose	Keys and important constraints
users	Login account for every person	PK user_id; UNIQUE username; CHECK role IN (admin, employee, customer)
customers	Customer profile	PK customer_id; FK user_id UNIQUE (1:1); UNIQUE customer_code, email; CHECK category
employees	Staff profile	PK employee_id; FK user_id UNIQUE; UNIQUE emp_code
utilities	Electricity / Water / Gas and their units	PK utility_id; UNIQUE name
tariff_plans	Fixed charge and tax for a utility + category	PK tariff_id; FK utility_id; UNIQUE (utility_id, category)
tariff_slabs	Rate per unit range	PK slab_id; FK tariff_id ON DELETE CASCADE; CHECK to_unit > from_unit
connections	A customer's service for one utility	PK; FK customer, utility, tariff; UNIQUE (customer_id, utility_id)
meters	Physical meter on a connection	PK; UNIQUE meter_no; CHECK status; initial_reading ≥ 0
meter_readings	Cumulative reading on a date	PK; FK meter_id; UNIQUE (meter_id, reading_date); CHECK consumption ≥ 0
bills	Bill for one reading (snapshot of charges)	PK; UNIQUE bill_no, reading_id (1:1 with reading); CHECK status
bill_items	Itemised lines of a bill	PK; FK bill_id ON DELETE CASCADE
payments	Money received against a bill	PK; UNIQUE receipt_no; CHECK amount > 0; CHECK method
complaints	Customer complaints	PK; FK customer, connection, assigned employee; CHECK priority, status
outages	Supply interruptions by area	PK; FK utility_id; CHECK status
notifications	In-app messages to a user	PK; FK user_id; index (user_id, is_read)
audit_log	Recent activity feed for the dashboard	PK log_id

Table 1: Tables in the schema (16)

1.2 Normalisation

- **1NF:** each column holds a single value. Things that repeat, like readings, payments and slabs, get their own table instead of extra columns.
- **2NF:** every table has a single-column primary key, so there are no partial dependencies. Where there is a natural composite key, such as (meter_id, reading_date), the other columns depend on all of it.
- **3NF / BCNF:** no non-key column depends on another non-key column. For example, rates aren't stored on `connections`; a connection points to a tariff plan, and the plan has its slabs. Units like kWh live only in `utilities`.
- **Where we broke 3NF on purpose:** `bills` keeps a copy of the units, charges, tax and total from the day it was made. Tariffs change, but a bill that has already gone out shouldn't. We also store `consumption` on each reading so a row makes sense on its own.

1.3 Triggers, views and indexes

Object	Type	Rule enforced
trg_reading_monotonic	BEFORE INSERT	Rejects a reading lower than the meter's highest reading (or initial reading).
trg_reading_date_order	BEFORE INSERT	Rejects a reading dated on/before the last reading date.
trg_no_overpay	BEFORE INSERT	Rejects a payment larger than the bill's outstanding amount.
trg_payment_status	AFTER INSERT	Adds to amount_paid, sets status paid/partial, notifies customer, logs activity.
trg_bill_notify	AFTER INSERT	Notifies the customer when a bill is generated.
trg_outage_notify	AFTER INSERT	Notifies every customer in the outage area who has that utility.
trg_complaint_status	AFTER UPDATE	Notifies the customer when complaint status changes.
v_bill_summary	View	Bill joined with connection, utility and customer, with outstanding amount.
v_overdue_bills	View	Unpaid bills past due date, with days overdue.
v_monthly_consumption	View	Total units and billed amount per utility per month (dashboard and reports).
6 indexes	B-tree	(meter_id, reading_date), (status, due_date), billing_month, area, complaint status, (user_id, is_read).

Table 2: Database objects that enforce business rules

Example – the overpayment trigger (from `schema.sql`):

```
CREATE TRIGGER trg_no_overpay
BEFORE INSERT ON payments
WHEN NEW.amount > (SELECT ROUND(total_amount - amount_paid, 2) FROM bills WHERE bill_id =
NEW.bill_id) + 0.001
BEGIN
    SELECT RAISE(ABORT, 'Payment exceeds outstanding balance');
END;
```

1.4 Billing formula

For each slab *i* with range [from_i, to_i) and rate r_i (the last slab has no upper limit):

$$\text{units in slab}_i = \max(0, \min(U, \text{to}_i) - \text{from}_i) \qquad \text{Energy} = \Sigma \text{ units in slab}_i \times r_i \qquad \text{Total} = (\text{Energy} + \text{Fixed}) \times (1 + \text{Tax\%} / 100)$$

Worked example (domestic electricity, 250 kWh): 100 × 3.50 = 350.00; 150 × 5.00 = 750.00; energy = 1,100.00; + fixed 50.00 = 1,150.00; + 5% tax 57.50 = **₹1,207.50** (verified by test UT-01).